AWS Services Decision Guide & Interview Cheat Sheet

Comprehensive study guide for AWS Solutions Architect and DevOps interviews

Storage

• S3 - files, backups, static websites. Use for object storage.

• EBS - block storage for a single EC2.

• EFS - shared file storage across multiple EC2s.

• Decision: Files->S3, Single Server Disk->EBS, Shared Files->EFS.

Real-world Example: *Choose the service based on the requirement rather than memorizing names.*

Databases

• RDS - relational SQL workloads.

• Aurora - high-performance MySQL/PostgreSQL compatible database.

• DynamoDB - NoSQL at massive scale.

• ElastiCache - caching layer.

• Redshift - analytics and warehousing.

• MemoryDB - Redis-compatible primary database.

• DMS - database migrations.

Real-world Example: *Choose the service based on the requirement rather than memorizing names.*

Compute

• EC2 - full server control.

• Lambda - event-driven serverless execution.

• ECS - AWS-native container orchestration.

• EKS - managed Kubernetes.

• Fargate - serverless containers.

• ECR - Docker image registry.

Real-world Example: *Choose the service based on the requirement rather than memorizing names.*

Security

• IAM Users for humans.

• IAM Roles for AWS services.

• KMS for encryption keys.

• Secrets Manager for passwords and API keys.

• Control Tower for multi-account governance.

Real-world Example: *Choose the service based on the requirement rather than memorizing names.*

Networking

• VPC for network isolation.

• ALB for HTTP/HTTPS routing.

• NLB for TCP/UDP workloads.

• Route 53 for DNS.

• CloudFront for CDN.

• API Gateway for APIs.

Real-world Example: *Choose the service based on the requirement rather than memorizing names.*

Monitoring

• CloudWatch: metrics, logs, alarms.

• CloudTrail: audit history.

• AWS Config: compliance and configuration tracking.

Real-world Example: *Choose the service based on the requirement rather than memorizing names.*

Messaging

• SNS for one-to-many notifications.

• SQS for queue-based processing.

• SES for email.

• EventBridge for event-driven workflows.

Real-world Example: *Choose the service based on the requirement rather than memorizing names.*

DevOps

• CloudFormation for Infrastructure as Code.

• Elastic Beanstalk for simple app deployment.

• Kinesis for real-time streaming.

• CodeCommit -> CodeBuild -> CodePipeline -> CodeDeploy for CI/CD.

Real-world Example: *Choose the service based on the requirement rather than memorizing names.*

Interview Decision Trees

- Need storage? S3 vs EBS vs EFS
- Need database? RDS/Aurora vs DynamoDB
- Need compute? EC2 vs Lambda
- Need containers? ECS vs EKS vs Fargate
- Need messaging? SNS vs SQS
- Need monitoring? CloudWatch vs CloudTrail vs Config
- Need networking? Route53 vs CloudFront vs API Gateway

Golden Rule

Always begin architecture discussions with: Based on the requirement... Then justify why one service is better than the alternatives.